

第2章 算法设计与优化

2.1 算法优化策略

有一些题目并不需要巧妙的思路和缜密的推理，就能找到一个解决方案，只是时间效率难以令人满意。降低时间复杂度的方法有很多，本小节的题目就覆盖了其中最常见的一些算法优化策略。

例2-1 【快速选择问题】谁在中间 (Who's in the Middle, POJ 2388)

给出 N ($1 \leq N < 10\,000$ 且 N 为奇数) 个大小不超过 10^6 的正整数，计算这些整数的中位数。

【代码实现】

```
// 陈锋
#include <cstdio>
#include <algorithm>
#include <cassert>
using namespace std;

const int NN = 1e5 + 4;
int N, A[NN];
int main() {
    scanf("%d", &N);
    for (int i = 0; i < N; i++) scanf("%d", &A[i]);
    int k = N / 2;
    nth_element(A, A + k, A + N);
    printf("%d\n", A[k]);
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》8.2.2 节快速排序、快速选择问题
注意：本题中对于 nth_element 函数的使用
*/
```

例2-2 【线性扫描；前缀和；单调性】子序列（Subsequence, SEERC 2006, UVa 1121)

有 n ($10 < n \leq 100\,000$) 个正整数组成一个序列，给定整数 S ($S < 10^9$)，求长度最短的连续序列，使它们的和大于或等于 S 。

【代码实现】

```
// 刘汝佳
#include <algorithm>
#include <cstdio>
using namespace std;

const int maxn = 1e5 + 8;
int A[maxn], B[maxn];
int main() {
    for (int n, S; scanf("%d%d", &n, &S) == 2 && n;) {
        for (int i = 1; i <= n; i++) scanf("%d", &A[i]);
        B[0] = 0;
        for (int i = 1; i <= n; i++) B[i] = B[i - 1] + A[i];
        int ans = n + 1, i = 1;
        for (int j = 1; j <= n; j++) {
            if (B[i - 1] > B[j] - S) continue; // (1) 没有满足条件的 i, 换下一个 j
            while (B[i] <= B[j] - S) i++; // (2) 求满足 B[i-1] <= B[j] - S 的最大 i
            ans = min(ans, j - i + 1);
        }
        printf("%d\n", ans == n + 1 ? 0 : ans);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 1.3 节例题 21
注意：本题中 for 嵌套 while 循环的时间复杂度是  $O(n)$ ，而不是  $O(n^2)$ 
同类题目：Garbage Heap, UVa 10755
        Feel Good, ACM/ICPC NEERC 2005, UVa 1619
*/
```

例2-3 【扫描；维护最大值】开放式学分制（Open Credit System, UVa 11078)

给定一个长度为 n 的整数序列 $\{A_0, A_1, \dots, A_{n-1}\}$ ($2 \leq n \leq 100\,000$)，找出两个整数 A_i 和 A_j ($i < j$)，使得 $A_i - A_j$ 尽量大。

【代码实现】

```
// 陈锋
#include <cassert>
#include <cstdio>
#include <algorithm>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
typedef long long LL;
const int MAXN = 1e5 + 4;
int A[MAXN];
int main() {
    int n, T;
    scanf("%d", &T);
    while (T--) {
        scanf("%d", &n);
        _for(i, 0, n) scanf("%d", &(A[i]));
        int m = A[0], ans = A[0] - A[1]; // maxA[i]
        _for(i, 1, n) // m = max{A0, A1, A_{i-1}}
            ans = max(m - A[i], ans), m = max(A[i], m);
        printf("%d\n", ans);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版1.3节例题18
同类题目：Cricket Field, ACM/ICPC NEERC 2002, UVa 1312
*/
```

例2-4 【Floyd判圈】计算器谜题（Calculator Conundrum, UVa 11549）

有一个老式计算器，只能显示 n 位数字。有一天，你无聊了，于是输入一个整数 k ($1 \leq n \leq 9$, $0 \leq k < 10^n$)，然后反复平方，直到溢出。每次溢出时，计算器会显示结果的最高 n 位和一个错误标记。然后，清除错误标记，继续平方。如果一直这样做下去，能得到的最大数是多少？比如，当 $n=1$, $k=6$ 时，计算器将依次显示6、3（36的最高位），9、8（81的最高位），6（64的最高位），3，…

【代码实现】

```

// 刘汝佳
#include <iostream>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
typedef long long LL;
LL K, M;
LL next(LL x) {
    LL ans = x * x;
    while (ans >= M) ans /= 10;
    return ans;
}
int main() {
    int T, n;
    scanf("%d", &T);
    while (T--) {
        scanf("%d%lld", &n, &K);
        M = 1;
        _for(i, 0, n) M *= 10;
        LL ans = K, k1 = K, k2 = K;
        do {
            k1 = next(k1), ans = max(ans, k1);
            k2 = next(k2), ans = max(ans, k2);
            k2 = next(k2), ans = max(ans, k2);
        } while (k1 != k2);
        printf("%lld\n", ans);
    }
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 1.3 节例题 19
*/

```

例2-5 【中途相遇】和为0的4个值（4 Values Whose Sum is Zero, ACM/ICPC SWERC 2005, UVa 1152）

给定4个包含 n ($1 \leq n \leq 4000$) 个元素的集合 A, B, C, D ，要求分别从中选取一个元素 a, b, c, d ，使得 $a+b+c+d=0$ 。问：有多少种选法？

例如， $A=\{-45,-41,-36,26,-32\}$ ， $B=\{22,-27,53,30,-38,-54\}$ ， $C=\{42,56,-37,-75,-10,-6\}$ ， $D=\{-16,30,77,-46,62,45\}$ ，则有5种选法： $(-45, -27, 42, 30)$ ， $(26, 30, -10, -46)$ ， $(-32, 22, 56,-46)$ ， $(-32, 30, -75, 77)$ ， $(-32, -54, 56, 30)$ 。

【代码实现】

```
// 陈锋
#include <cstdio>
#include <algorithm>
using namespace std;

const int NN = 4000 + 8;
int A[NN], B[NN], C[NN], D[NN], sums[NN * NN];
int main() {
    int T, n, c;
    scanf("%d", &T);
    while (T--) {
        scanf("%d", &n);
        for (int i = 0; i < n; i++)
            scanf("%d%d%d%d", &A[i], &B[i], &C[i], &D[i]);
        c = 0;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                sums[c++] = A[i] + B[j];
        sort(sums, sums + c);
        long long cnt = 0;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) {
                pair<int*, int*> p = equal_range(sums, sums + c, -C[i] - D[j]);
                cnt += p.second - p.first;
            }
        printf("%lld\n", cnt);
        if (T) printf("\n");
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 8-3
注意：如何通过对 equal_range 的一次调用获得目标区间的两个端点
同类题目：Jurassic Remains, Codeforces Gym 101388J
          Non-boring sequences, UVa 1608
          Weak Key, ACM/ICPC Seoul 2004, UVa 1618
*/
```

例2-6 【滑动窗口】唯一的雪花（Unique Snowflakes, UVa 11572）

输入一个长度为 n ($n \leq 10^6$) 的序列 A ，找到一个尽量长的连续子序列 $A_L \sim A_R$ ，使得该序列中没有相同的元素。

【代码实现】

```
// 刘汝佳
#include <bits/stdc++.h>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
typedef long long LL;
const int MAXN = 1e6 + 4;
int A[MAXN];
int main() {
    int T, n;
    scanf("%d", &T);
    while (T--) {
        scanf("%d", &n);
        _for(i, 0, n) scanf("%d", &(A[i]));
        int L = 0, R = 1, ans = 1;          // [L, R), 滑动窗口长度最大值
        set<int> s = {A[L]};                // 当前窗口中的所有元素, C++11 的容器初始化语法
        while (R < n) {
            while (s.count(A[R]) && L < R) s.erase(A[L++]);
            s.insert(A[R++]);
            ans = max(ans, R - L);
        }
        printf("%d\n", ans);
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》例题 8-7
同类题目: Shuffle, UVa 12174
          Smallest Sub-Array, UVa 11536
*/
```

例2-7 【滑动窗口；单调队列】滑动窗口 (Sliding Window, POJ 2823)

输入正整数 k 和一个长度为 n 的整数序列 $\{A_1, A_2, A_3, \dots, A_n\}$ 。定义 $f(i)$ 表示从元素 i 开始的连续 k 个元素的最小值，即 $f(i) = \min\{A_i, A_{i+1}, \dots, A_{i+k-1}\}$ 。要求计算 $f(1), f(2), f(3), \dots, f(n-k+1)$ 。例如，对于序列 $\{5, 2, 6, 8, 10, 7, 4\}$ ， $k=4$ ，则 $f(1)=2, f(2)=2, f(3)=6, f(4)=4$ 。

【代码实现】

```

// 陈锋
#include <cstdio>
#include <deque>
#include <iostream>
#include <queue>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
const int NN = 1e6 + 8;
int N, K, A[NN], Q[NN];
int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    cin >> N >> K;
    _for(i, 0, N) cin >> A[i];
    int head = 0, tail = 0;
    _for(i, 0, N) {
        while (head < tail && Q[head] <= i - K) ++head;
        while (head < tail && A[Q[tail - 1]] >= A[i]) --tail;
        Q[tail++] = i;
        if (i >= K - 1) cout << A[Q[head]] << (i == N - 1 ? "\n" : " ");
    }

    head = 0, tail = 0;
    _for(i, 0, N) { // 单调递减队列，顶端是最大元素
        while (head < tail && Q[head] <= i - K) ++head;
        while (head < tail && A[Q[tail - 1]] <= A[i]) --tail;
        Q[tail++] = i;
        if (i >= K - 1) cout << A[Q[head]] << (i == N - 1 ? "\n" : " ");
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》8.5节滑动窗口部分
注意：本题队列使用左闭右开区间，必须使用手写的队列而不是 deque，才能够通过时限
同类题目：Robotruck, UVa 1169
*/

```

例2-8 【线性扫描；事件点处理】流星（Meteor, Seoul 2007, UVa 1398）

给你一个矩形照相机，还有 n ($1 \leq n \leq 100\,000$) 个流星的初始位置和速度，求能照到流星最多的时刻。注意，在相机边界上的点不会被照到。如图2-1所示，流星2,3,4,5将不会被照到，因为它们从来没有经过图中矩形的内部。

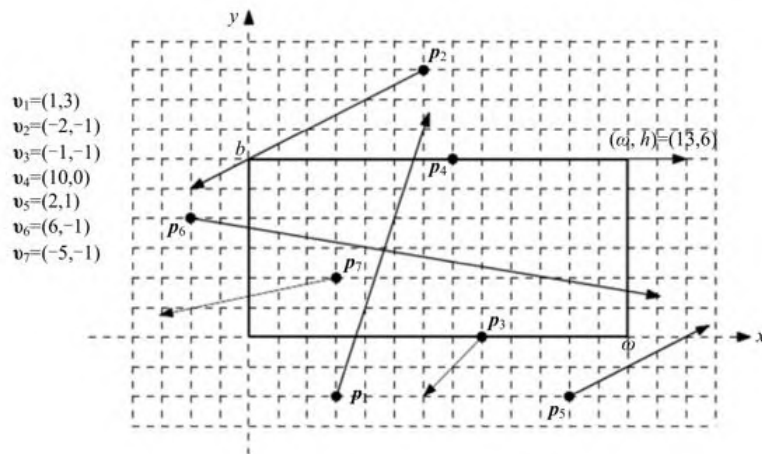


图2-1 流星

相机的左下角为(0,0)，右上角为(w,h) ($1 \leq w, h \leq 100\ 000$)。每个流星用两个向量 $\mathbf{p}$ 和 $\mathbf{v}$ 表示。其中， $\mathbf{p}$ 为初始 ($t=0$ 时) 位置， $\mathbf{v}$ 为速度。在时刻 t ($t \geq 0$) 的位置是 $\mathbf{p} + t\mathbf{v}$ 。比如，若 $\mathbf{p}=(1,3)$ ， $\mathbf{v}=(-2,5)$ ，则 $t=0.5$ 时该流星的位置为 $(1,3) + 0.5 \times (-2,5)=(0, 5.5)$ 。

【代码实现】


```

// 刘汝佳
#include <cstdio>
#include <algorithm>
using namespace std;
// 0<x+at<w
void update(int x, int a, int w, double& L, double& R) {
    if(a == 0) {
        if(x <= 0 || x >= w) R = L - 1; // 无解
    } else if(a > 0) {
        L = max(L, -(double)x / a);
        R = min(R, (double)(w - x)/a);
    } else {
        L = max(L, (double)(w - x)/a);
        R = min(R, -(double)x / a);
    }
}

const int maxn = 100000 + 10;

struct Event {
    double x;
    int type;
    bool operator < (const Event& a) const {
        return x < a.x || (x == a.x && type > a.type); // 先处理右端点
    }
} events[maxn*2];

int main() {
    int T;
    scanf("%d", &T);
    while(T--) {
        int w, h, n, e = 0;
        scanf("%d%d%d", &w, &h, &n);
        for(int i = 0; i < n; i++) {
            int x, y, a, b;
            scanf("%d%d%d%d", &x, &y, &a, &b);
            // 0 < x + at < w, 0 < y + bt < h, t>=0
            double L = 0, R = 1e9;
            update(x, a, w, L, R);
            update(y, b, h, L, R);
            if(R > L) {
                events[e++] = (Event){L, 0};
                events[e++] = (Event){R, 1};
            }
        }
        sort(events, events+e);
        int cnt = 0, ans = 0;
        for(int i = 0; i < e; i++) {
            if(events[i].type == 0) ans = max(ans, ++cnt);
            else cnt--;
        }
        printf("%d\n", ans);
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 1.3 节例题 20
同类题目: Distant Galaxy, UVa 1382
          Cave, UVa 1442
*/

```

另外，本题还可以完全避免实数运算，全部采用整数。只需要把代码中的double全部改成int，然后在update函数中把所有返回值乘以 $lcm(1,2,\dots,10)=2520$ 即可（想一想，为什么）。

```
void update(int x, int a, int w, int& L, int& R) {
    if(a == 0) {
        if(x <= 0 || x >= w) R = L - 1;           // 无解
    } else if(a > 0) {
        L = max(L, -x * 2520 / a);
        R = min(R, (w - x) * 2520 / a);
    } else {
        L = max(L, (w - x) * 2520 / a);
        R = min(R, -x * 2520 / a);
    }
}
```

例2-9 【逆序对数计算】参谋 (Brainman, POJ 1804)

给出长度为 N 的数组，计算通过多少次相邻元素的交换操作，可以将其变为升序。

【代码实现】

```

// 陈锋
#include <algorithm>
#include <cassert>
#include <cstdio>
using namespace std;

const int NN = 1000 + 4;
int N, A[NN], B[NN];
int merge(int l, int r) { // 归并排序, 求 A[l,r) 的逆序对数
    if (r - l <= 1) return 0;
    int mid = (l + r) / 2, ans = merge(l, mid) + merge(mid, r);
    copy(A + l, A + r, B + l); // A[l,r) -> B[l,r)
    int a = l, b = mid, i = l;
    while (a < mid || b < r) {
        if ((a < mid && B[a] <= B[b]) || b >= r)
            A[i++] = B[a++];
        else // 左边所有比 B[b] 大的数分别与 B[b] 构成逆序对
            ans += mid - a, A[i++] = B[b++];
    }
    return ans;
}

int main() {
    int T = 0;
    scanf("%d", &T);
    for (int k = 1; k <= T; k++) {
        if (k > 1) printf("\n");
        scanf("%d", &N);
        for (int i = 0; i < N; i++) scanf("%d", &A[i]);
        int ans = merge(0, N);
        printf("Scenario #%d:\n%d\n", k, ans);
    }

    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》8.2.1 节归并排序、逆序对问题
注意: 本题中归并排序的合并部分仅仅使用了一个循环
同类题目: Frosh Week, HDU 3743
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点, 如表2-1所示。

表2-1 算法优化策略例题归纳